

Claude Code 最佳实践：从「能用」到「用得好」的 15 个关键技巧

AIReadingHub 2026-04-19  186  阅读9分钟[关注](#)

最近 GitHub 上有个仓库火了——**claude-code-best-practice**，4.6 万 star，登顶 GitHub Trending 第一。

作者 Shayan Rais 把 Claude Code 的用法从底层拆透了，涵盖记忆系统、子代理、技能模块、Hook 机制等全链路。我花了几个小时通读了整个仓库，把最有价值的部分提炼出来，结合我自己的使用经验，整理成这篇实操指南。

如果你已经在用 **Claude Code**，这篇文章至少能让你的效率再翻一倍。

一、大多数人用错的第一件事：上下文管理

Claude Code 现在支持 100 万 token 的上下文窗口，听起来很大，但实际上——

在 **30-40 万 token** 时，模型表现就开始退化了。

这是 **claude-code-best-practice** 仓库里提到的一个核心概念：**Context Rot**（上下文腐烂）。模型并不会在塞满 100 万 token 之后才变差，而是远远提前就开始丢精度、忘指令、漏细节。

怎么做？

手动 **/compact**，不要等系统自动触发。

系统自动压缩触发时，上下文已经在退化区间了。正确做法是在 **50% 左右**就手动执行 **/compact**。

更好的做法：带提示词压缩。

比如输入 `/compact` 保留 `auth` 重构的上下文，丢掉之前调试测试的部分。

有针对性地告诉 Claude 保留什么、丢弃什么，压缩质量远好于盲压。

还有一个反直觉的操作：Rewind 优于修正

当 Claude 的一次操作出了问题，很多人的本能反应是在对话里纠正它。但更好的做法是：按两次 **Esc** 回退到出错之前的节点，然后重新给指令。

原因很简单：失败的尝试会留在上下文里，持续污染后续的判断。回退到干净的状态重来，比在脏上下文里修补更高效。

二、CLAUDE.md 文件：你的 AI 工程师"入职手册"

CLAUDE.md 是 Claude Code 的记忆核心——每次启动时自动加载的指令文件。但大多数人要么不写，要么写成了一坨无用的长文档。

最佳实践：控制在 200 行以内

Claude Code 创始人 Boris Cherny 在多次访谈中反复强调这个数字。HumanLayer 团队甚至把他们的 CLAUDE.md 压缩到了 60 行。

为什么？指令越长，被忽略的概率越高。模型会在大量文本中"走神"。

核心原则："任何开发者启动 Claude，说'跑测试'，第一次就能成功"

这是 Boris 给出的 CLAUDE.md 黄金标准。你的 CLAUDE.md 应该包含：

- 项目的构建和测试命令
- 代码风格和约定（简洁版）
- 必须遵守的硬性规则

不应该包含： 可以从代码本身推断出来的信息，比如目录结构、函数列表、架构描述。Claude 可以自己读代码。

防止指令被忽略的技巧

当你发现 Claude 总是无视某条规则时，用 XML 标签包裹：

ini

 体验AI代码助手

 代码解读

复制代码

```
1 <important if="editing API routes">
2 所有 API 路由必须包含 rate limiting 中间件
3 </important>
```

条件触发式的指令比全局的"你必须做 X"更容易被遵守。

Monorepo 场景

在 monorepo 中，可以在不同子目录放不同的 CLAUDE.md。Claude Code 会自动加载：

- 祖先目录的 CLAUDE.md（向上查找）
- 当前目录的 CLAUDE.md（立即加载）
- 子目录的 CLAUDE.md（按需懒加载）

这样前端、后端、基础设施可以有各自的规则，互不干扰。

三、子代理（Subagents）：让 Claude 自己分工协作

这是 Claude Code 最被低估的功能之一。

子代理 = 拥有独立上下文的自主执行单元。 每个子代理可以有自己的工具权限、模型选择、记忆设置。

什么时候用子代理？

一句话判断："这个工具的输出，我后续还需要原始数据，还是只需要结论？"

如果只需要结论——交给子代理。它在隔离上下文中处理完，返回一个精简结果，不污染主对话。

实战场景

▼ yaml 体验AI代码助手 代码解读 复制代码

```
1 # .claude/agents/code-reviewer.md
2 ---
3 name: code-reviewer
4 description: 审查代码变更，关注安全和性能
5 model: opus
6 tools: Read, Grep, Glob
7 ---
8
9 审查以下变更，重点关注：
10 1. 安全漏洞（注入、XSS、敏感信息泄露）
11 2. 性能问题（N+1 查询、不必要的循环）
12 3. 边界条件处理
```

你可以同时启动多个子代理并行处理不同任务——一个跑测试，一个审查代码，一个搜索相关文件——效率直接翻倍。

四、技能系统（Skills）：教 Claude 学会你的业务

Skills 是 Claude Code 中最灵活的模块化知识单元。它是一个文件夹，而不只是一个文件。

▼ bash 体验AI代码助手 代码解读 复制代码

```
1 .claude/skills/deploy-checker/
2 └─ SKILL.md           # 技能定义和触发条件
3 └─ references/        # 参考文档
4 └─ scripts/          # 可执行脚本
5 └─ examples/         # 示例
```

关键经验

1. 技能描述是"触发器"，不是"摘要"

错误写法： 这个技能用于部署检查

正确写法： 当用户要求部署、发布、上线、推送到生产环境时使用此技能

描述的作用是让 Claude 判断"什么时候该激活我"，所以要用触发场景来写。

2. 给目标和约束，不给步骤

不要在技能里写"第一步做 X，第二步做 Y"。Claude 不需要被手把手教。

写清楚目标是什么、边界在哪里就够了。过度指导反而会限制模型找到更优解的能力。

3. 建一个"踩坑记录"区域

在每个 SKILL.md 里加一个 Gotchas 部分，记录已知的坑。比如：

▼ markdown  体验AI代码助手  代码解读 复制代码

```
1 ## Gotchas
2 - 生产环境的数据库连接超时设置是 5s，不是默认的 30s
3 - iOS 端的推送证书每年 1 月需要更新
4 - staging 环境的 CDN 缓存是 1 小时，调试时记得清除
```

4. 用 `!command` 注入动态上下文

在 SKILL.md 中嵌入 shell 命令，技能加载时自动执行并注入结果：

▼ markdown  体验AI代码助手  代码解读 复制代码

```
1 当前 Git 分支和最近提交：
2 `!git log --oneline -5`
```

这样技能每次被触发时，都能拿到实时信息。

五、Hooks：在 AI 执行循环之外加入你的规则

Hooks 是在 Claude 的 agentic 循环之外运行的事件处理器，支持 25 种事件。

实战例子：自动格式化

设置 `PostToolUse` Hook，在 Claude 每次写完文件后自动运行 `prettier` 或 `eslint --fix`。Claude 不需要记住格式化规则，Hook 自动处理。

实战例子：安全守卫

设置 **PreToolUse** Hook，拦截危险命令：

▼ json 体验AI代码助手 代码解读 复制代码

```
1  {
2    "hooks": {
3      "PreToolUse": [{
4        "matcher": "Bash",
5        "command": "检查命令是否包含 rm -rf, git push --force 等危险操作"
6      }]
7    }
8  }
```

最酷的用法：Stop Hook

在 Claude 准备结束一轮对话时触发，可以让它：

- 检查是否真的完成了所有任务
- 运行一遍验证测试
- 自动生成总结

六、工作流：高手都在用的开发模式

Plan → Execute → Review

Claude Code 创始人的工作流核心：永远从 **Plan Mode** 开始。

不要上来就让 Claude 写代码。先让它分析需求、拆分任务、输出计划。你确认计划后，再进入执行。

更进阶的做法：

"先面试我，用 **AskUserQuestion** 工具问清楚所有细节，然后在新会话中执行。"

这样做的好处是：执行会话的上下文里只有干净的需求和计划，没有来回讨论的噪音。

小 PR 原则

Boris Cherny 的数据：一天 **141 个 PR**，中位数 **118 行**。

大 PR 会让审查者疲劳，也会让 Claude 在处理时更容易出错。把大任务拆成一系列小的、可独立验证的 PR。

用第二个 Claude 审查第一个 Claude 的计划

启动一个子代理，角色设定为"资深工程师"，让它审查主会话中 Claude 的实现计划。两个独立的上下文互相检查，能抓到很多单一视角看不到的问题。

七、权限管理：安全与效率的平衡

很多人为了省事，开启 `--dangerously-skip-permissions`。这是个坏主意。

更好的替代方案

1. 通配符权限

在 settings.json 中配置：

▼ json 体验AI代码助手 代码解读 复制代码

```
1 {
2   "permissions": {
3     "allow": [
4       "Bash(npm run *)",
5       "Bash(git status)",
6       "Bash(git diff *)"
7     ]
8   }
9 }
```

只放行你信任的命令模式，而不是全部跳过。

2. /sandbox 模式

数据显示 `/sandbox` 能减少 **84% 的权限弹窗**，同时保持安全边界。

3. Auto Mode

基于模型的安全分类器，自动判断操作是否安全。比全跳过安全得多，比手动确认方便得多。

八、搜索策略：为什么 Claude Code 放弃了 RAG

一个有趣的事实：Claude Code 团队曾经尝试过向量数据库和 RAG 方案，**最终放弃了**。

他们发现 **agentic search**（基于 `glob + grep` 的主动搜索）效果更好。

原因是模型可以根据搜索结果动态调整下一步搜索策略，而 RAG 的一次性检索往往不够精确。

这对你的启示是：**不需要给 Claude Code 建索引或嵌入数据库**，它自带的文件搜索能力已经是最优解。

九、产品验证技能："值得花一周来打磨"

Boris 特别强调的一个方向：为你的产品构建专门的验证技能。

比如：

- `signup-flow-driver` ——自动走完注册流程，检查每一步
- `checkout-verifier` ——模拟下单到支付的全链路
- `api-health-checker` ——验证所有 API 端点的响应

这类技能的投资回报极高。写一次，每次改动后自动验证，省掉大量手动测试时间。

十、正在改变游戏规则的新功能

claude-code-best-practice 仓库还追踪了一些前沿功能：

- **Agent Teams**：多个代理并行在同一代码库上工作
- **Routines**：在 Anthropic 云端自动执行任务（类似 GitHub Actions 但由 AI 驱动）
- **Chrome 扩展**：直接在浏览器里用 Claude Code 调试前端
- **Voice Dictation**：20 种语言的语音输入，边说边写代码
- **Agent SDK**：用 Python/TypeScript 构建生产级 AI 代理的官方 SDK

写在最后

Claude Code 的能力边界远超大多数人的使用深度。上面这些实践，核心逻辑只有一条：

把 **Claude** 当成一个需要好的工作环境的工程师，而不是一个需要精确指令的工具。

给它清晰的上下文（CLAUDE.md），合理的分工（子代理），可复用的知识（技能），自动化的流程（Hooks），以及安全的边界（权限管理）。

GitHub 仓库地址：**shanraishan/claude-code-best-practice**，持续更新中，建议 Star 收藏。

你在用 Claude Code 时踩过什么坑？或者有什么独家技巧？欢迎在评论区分享。

标签： Claude 人工智能

评论 0



登录 / 注册 即可发布评论!





暂无评论数据

目录

收起 ^

一、大多数人用错的第一件事：上下文管理

怎么做？

还有一个反直觉的操作：Rewind 优于修正

二、CLAUDE.md 文件：你的 AI 工程师"入职手册"

最佳实践：控制在 200 行以内

核心原则："任何开发者启动 Claude，说'跑测试'，第一次就能成功"

防止指令被忽略的技巧

Monorepo 场景

三、子代理（Subagents）：让 Claude 自己分工协作

什么时候用子代理？

实战场景

四、技能系统（Skills）：教 Claude 学会你的业务

关键经验

五、Hooks：在 AI 执行循环之外加入你的规则

实战例子：自动格式化

实战例子：安全守卫

最酷的用法：Stop Hook

六、工作流：高手都在用的开发模式

Plan → Execute → Review

小 PR 原则

用第二个 Claude 审查第一个 Claude 的计划

七、权限管理：安全与效率的平衡

更好的替代方案

八、搜索策略：为什么 Claude Code 放弃了 RAG

九、产品验证技能："值得花一周来打磨"

十、正在改变游戏规则的新功能

写在最后

相关推荐

让 AI 帮你写大数据AI开发代码：MaxFrame Coding Skill 正式发布

70阅读 · 0点赞

给 Claude Code 加上 Windows 提醒——一个小功能，少操十份心

89阅读 · 0点赞

第九章：我是如何剖析 Claude Code 的 CLI 里的安全沙盒与指令拦截机制的

181阅读 · 4点赞

2026年最新谷歌Gmail邮箱注册后无法登陆Gemini怎么办？谷歌账号打不开Gemini提示出了点问题的终极解...

211阅读 · 0点赞

Karpathy用一个文件拿了3万星：AI写代码的三个致命问题，终于有人说清楚了

82阅读 · 1点赞

精选内容

Generalist之后，罗剑岚团队推出LWD，也要变革具身智能训练范式

机器之心 · 16阅读 · 0点赞

一觉醒来，大模型就帮我排查完页面性能问题

candyTong · 112阅读 · 3点赞

我给 AI 做了场入职培训

AI驯兽师Grace · 70阅读 · 2点赞

深入浅出 LangChain —— 第三章：模型抽象层

深海鱼在掘金 · 68阅读 · 1点赞

Vibe Usage：1个月迭代总结，一个用户要什么就做什么的产品实践

阴明 · 48阅读 · 0点赞

为你推荐

SQL 优化的 15 个最佳实践

小人物代码 | 2年前 | 👁 170 | 👍 点赞 | 💬 1

MySQL

VS Code 代码片段指南: 从基础到高级技巧

Immerse | 1年前 | 👁 1.1k | 👍 7 | 💬 评论

Visual ...

代码规范

敏捷开发

生成网页：AI 直接把「低代码」干掉了

冲向超级个体 | 1年前 | 👁 3.8k | 👍 20 | 💬 10

前端

AIGC

Claude

提升效率！Linux 管理员必用的10个关键技巧

编程学习网 | 4年前 | 👁 790 | 👍 2 | 💬 评论

Linux

后端

分享我在前端高速成长的八个阶段

小明大白菜 | 1年前 | 👁 802 | 👍 16 | 💬 3

前端

《Python编程：从入门到实践》高清pdf 百度网盘

| 2年前 | 👁 2.9k | 👍 点赞 | 💬 评论

Python

聊聊sql优化的15个小技巧

Java工程师的修炼之道 | 4年前 | 👁 1.5k | 👍 8 | 💬 评论

后端

用好语言模型：temperature、top-p等核心参数解析

Baihai_IDP | 2年前 | 👁 3.2k | 👍 10 | 💬 5

人工智能

AIGC

LLM

6 个实用的 Code Review 实践技巧

码农突围 | 5年前 | 👁 361 | 👍 点赞 | 💬 评论

Java

Code Review最佳实践

简栈文化 | 6年前 | 👁 452 | 👍 点赞 | 💬 评论

源码

史上最详细的使用Laf接入Claude-api教程

xintianyou | 2年前 | 👁 20k | 👍 21 | 💬 55

前端

Claude

从失望到精通：AI 大模型的掌握与运用技巧

悟鸣 | 2年前 | 👁 547 | 👍 1 | 💬 评论

算法

深度好文！中国GPT用户的第三阶段：揭秘你不知道的道与术

豆哥AI读书 | 2年前 | 👁 150 | 👍 点赞 | 💬 评论

ChatGPT

从 MLOps 到 LMOps 的关键技术嬗变

百度Geek说 | 2年前 | 796 | 1 | 评论

人工智能

101个非常棒的 GitHub仓库，绝对有你用的到的! 🤖

geekskai_com | 5年前 | 1.9k | 2 | 4

GitHub

